Trabajo Práctico Nro. 2

Digitalización de infraestructuras tecnológicas

Algoritmos y Estructuras de Datos

Curso: K1021

Profesor: Ing. Diego Juan

Año 2026

Condición de aprobación del TP

Para aprobar el trabajo práctico se deben realizar correctamente todos los requerimientos propuestos, utilizando el lenguaje C/C++ visto en las clases teóricas. Se deben cumplir las fechas de entrega; no cumplirlas significa quedar fuera de la carrera por la Aprobación Directa. Una vez realizada la entrega, se citará al equipo para hacer un coloquio individual sobre la entrega, el cual debe aprobarse.
Se debe entregar una carpeta o folio al docente del curso (una por equipo) que cuente con:
Una carátula con los integrantes del equipo y el código de grupo asignado.
Los algoritmos desarrollados y explicados en código fuente y/o diagramas.

Contexto del proyecto

En el marco de la digitalización y optimización de infraestructuras tecnológicas, la Dirección de Sistemas ha iniciado un programa para auditar los recursos de hardware distribuidos en los distintos centros de procesamiento de datos. Se requiere una aplicación que permita procesar la información recolectada de los equipos y mostrar resultados estadísticos sobre el material inventariado.

Requerimientos

Se debe desarrollar una aplicación que despliegue un menú con las siguientes opciones:

1. Cargar y procesar datos de componentes.
2. Mostrar resultados
3. Salir

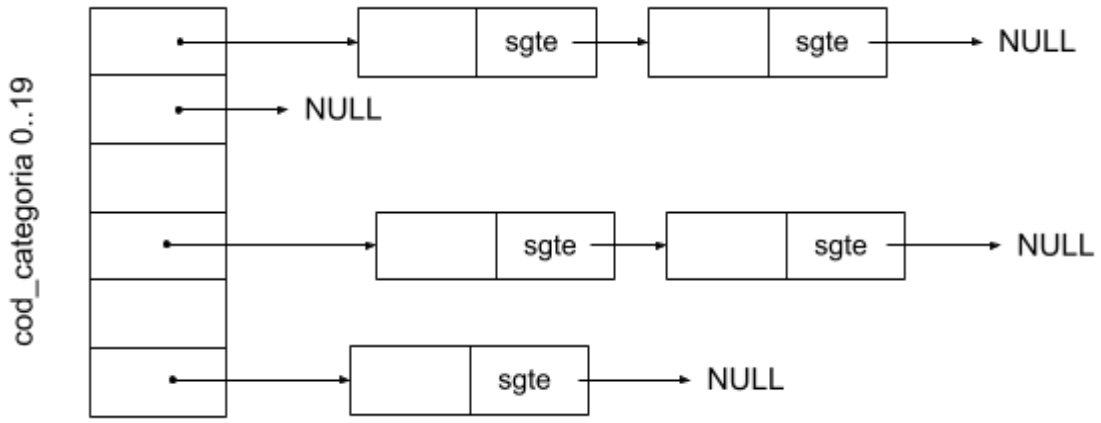
Descripción funcional

Si el usuario elige la opción "Cargar y procesar datos de componentes", la aplicación deberá invocar a un módulo o función encargada de realizar esta tarea. A su vez, este módulo podrá llamar a otros para efectuar tareas específicas de manera modular.
Los datos deben ser leídos desde un archivo binario. El archivo puede estar previamente generado o bien ser creado mediante una función auxiliar del programa. Cada registro del archivo contiene la siguiente información por componente:

- ID de componente (entero, identificador único)
- Tipo de componente (cadena de hasta 50 caracteres)
- Código de categoría (entero de 0 a 19)
- Modelo del dispositivo (cadena de hasta 50 caracteres)
- Fabricante / Marca (cadena de hasta 50 caracteres)
- Consumo energético en Watts (entero)

La información leída del archivo se debe organizar en un **vector de punteros a listas simplemente enlazadas**, donde cada posición del vector representa una categoría de hardware determinada (de 0 a 19). Cada puntero apunta al primer nodo de una lista que contiene los componentes correspondientes a dicha categoría.
Cada código de categoría puede representar a una CPU, GPU, Disco SSD , Memoria RAM, Fuente de alimentación, entre otros.

El vector debe ser declarado como un array de punteros conforme a la estructura de listas:
NodoComponente* vectorCategorias[CANT_CATEGORIAS];



La manipulación de las listas simplemente enlazadas debe realizarse exclusivamente utilizando las funciones del TAD Lista implementado y visto durante el curso, haciendo las adaptaciones que correspondan. Entre estas funciones se incluye obligatoriamente insertarOrdenado, utilizada para insertar cada componente en su respectiva lista manteniendo un criterio de orden ascendente según su ID de componente.

Si el usuario elige la opción "Mostrar resultados", la aplicación deberá invocar a una función que reciba por parámetro el vector de punteros a listas, procese los datos y muestre por pantalla:

- Cantidad total de componentes registrados en el sistema.
- Promedio de consumo energético (en Watts) por categoría.
- Categoría con mayor cantidad de componentes inventariados.
- Cantidad de componentes por cada categoría.
- Fabricante más frecuente en el registro.
- Modelo y ID del componente con mayor consumo energético.

Si el usuario elige la opción "Salir", la aplicación debe finalizar correctamente liberando los recursos de memoria necesarios.

Consideraciones técnicas

Bibliotecas: Uso de <cstdio> (para fopen, fread, fwrite, etc.) y <cstring> (para manejo de char[]). Se permite <iostream> para cin y cout. Memoria dinámica: La creación y liberación de nodos debe realizarse estrictamente con los operadores new y delete.